

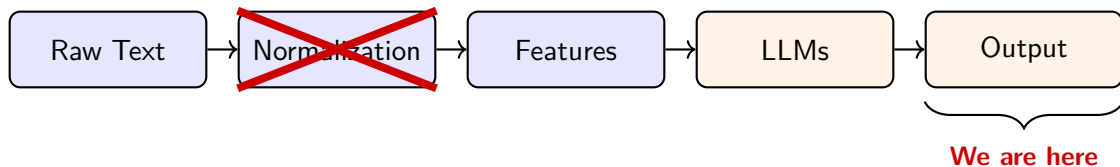
Textual Analysis in Finance

Week 6: Post-training adaptation

Tri, Minh Phan

Faculty of Business and Economics
University of Basel

Where are we?



Why post-training adaptation?

- ▶ Pre-trained LLMs (ChatGPT, Claude, etc.) are powerful, but **general-purpose**
- ▶ Training (or fine-tuning) a full LLM for **specific tasks**—e.g., financial analysis, economic reasoning—is **computationally expensive** and often **infeasible**
- ▶ How can we adapt LLMs to specific tasks without retraining?
 - ▶ *In-context learning*: no training required
 - ▶ *Quantization*: train the model but reduce memory and computation
 - ▶ *Low-Rank Adaptation (LoRA)*: train the model but only update a small set of parameters
 - ▶ *Retrieval-Augmented Generation (RAG)*: In-context learning + contextualized prompts

In-context learning

What is In-context learning?

- ▶ ~~In-context learning~~, a.k.a, the way people use ChatGPT
- ▶ $p(w_{t+1} | w_{1:t}, \text{prompt_examples}) \rightarrow$ we give ChatGPT some examples in the prompt, it will adapt the answers accordingly
- ▶ If the prompt = giving three examples $\rightarrow$ **three-shot learning**. E.g.,
News: "Company X reports record earnings." Sentiment: Positive
News: "Stock Y falls after CEO resignation." Sentiment: Negative
News: "Economic growth remains steady." Sentiment: Neutral
News: "Bank Z announces dividend cut." Sentiment:
- ▶ If the prompt = instructions $\rightarrow$ **zero-shot learning**. E.g.,
News: "Bank Z announces dividend cut." Sentiment:
- ▶ **Advantages:** no training, fast, flexible, few-shot
- ▶ **Limitations:** prompt-sensitive, token-limited, knowledge-static

Quantization

What is Quantization?

- ▶ In-context learning doesn't require re-training LLMs → their knowledge is still **not updated for specific domains**
 - ▶ What if re-training is needed but costs are too high?

What is Quantization?

- ▶ In-context learning doesn't require re-training LLMs → their knowledge is still **not updated for specific domains**
 - ▶ What if re-training is needed but costs are too high?
- ▶ **Quantization** and **Low-Rank Adaptation**: efficient ways to adapt models

What is Quantization?

- ▶ In-context learning doesn't require re-training LLMs → their knowledge is still **not updated for specific domains**
 - ▶ What if re-training is needed but costs are too high?
- ▶ **Quantization** and **Low-Rank Adaptation**: efficient ways to adapt models
- ▶ “Adapt” means updating the LLM's pre-trained parameters to fit specific tasks

What is Quantization?

- ▶ In-context learning doesn't require re-training LLMs → their knowledge is still **not updated for specific domains**
 - ▶ What if re-training is needed but costs are too high?
- ▶ **Quantization** and **Low-Rank Adaptation**: efficient ways to adapt models
- ▶ “Adapt” means updating the LLM's pre-trained parameters to fit specific tasks
- ▶ Ultimately, an LLM's parameters are just matrices of decimal numbers (*floats*)

What is Quantization?

- ▶ In-context learning doesn't require re-training LLMs → their knowledge is still **not updated for specific domains**
 - ▶ What if re-training is needed but costs are too high?
- ▶ **Quantization** and **Low-Rank Adaptation**: efficient ways to adapt models
- ▶ “Adapt” means updating the LLM's pre-trained parameters to fit specific tasks
- ▶ Ultimately, an LLM's parameters are just matrices of decimal numbers (*floats*)
- ▶ At a high level, quantization = reducing number precision (like rounding)

What is Quantization?

- ▶ In-context learning doesn't require re-training LLMs → their knowledge is still **not updated for specific domains**
 - ▶ What if re-training is needed but costs are too high?
- ▶ **Quantization** and **Low-Rank Adaptation**: efficient ways to adapt models
- ▶ “Adapt” means updating the LLM's pre-trained parameters to fit specific tasks
- ▶ Ultimately, an LLM's parameters are just matrices of decimal numbers (*floats*)
- ▶ At a high level, quantization = reducing number precision (like rounding)
- ▶ Quantization asks: Do we really need '0.1234567890123456' (64-bit) for the model to perform well, or is only '0.1235' (16-bit) sufficient?

What is Quantization?

- ▶ In-context learning doesn't require re-training LLMs → their knowledge is still **not updated for specific domains**
 - ▶ What if re-training is needed but costs are too high?
- ▶ **Quantization** and **Low-Rank Adaptation**: efficient ways to adapt models
- ▶ “Adapt” means updating the LLM's pre-trained parameters to fit specific tasks
- ▶ Ultimately, an LLM's parameters are just matrices of decimal numbers (*floats*)
- ▶ At a high level, quantization = reducing number precision (like rounding)
- ▶ Quantization asks: Do we really need '0.1234567890123456' (64-bit) for the model to perform well, or is only '0.1235' (16-bit) sufficient?
- ▶ Procedure:
 - ▶ Choose a pre-trained LLM (e.g., [Mistral-7B](#), [Meta-LLaMA-3-70B](#))
 - ▶ Quantization step: convert model weights from high precision (e.g., 32/64-bit) to lower precision (e.g., 16-bit, 8-bit, 4-bit)
 - ▶ Re-train the quantized model to reduce accuracy loss

Low-rank Adaptation

What is Low-rank Adaptation (LoRA)?

- ▶ Introduced by Hu et al. [2022], this represents a paradigm shift in how we adapt models (and I think this is a **genius** idea!)
- ▶ An LLM's weights are stored in massive matrices. Let $W \in \mathbb{R}^{d \times k}$ represent a single weight matrix (e.g., in an Attention layer).

The “Small” Case: GPT-2 Medium

Consider the Query weight matrix W_q where $d = k = 1024$:

$$1,024 \times 1,024 = \mathbf{1,048,576} \text{ parameters}$$

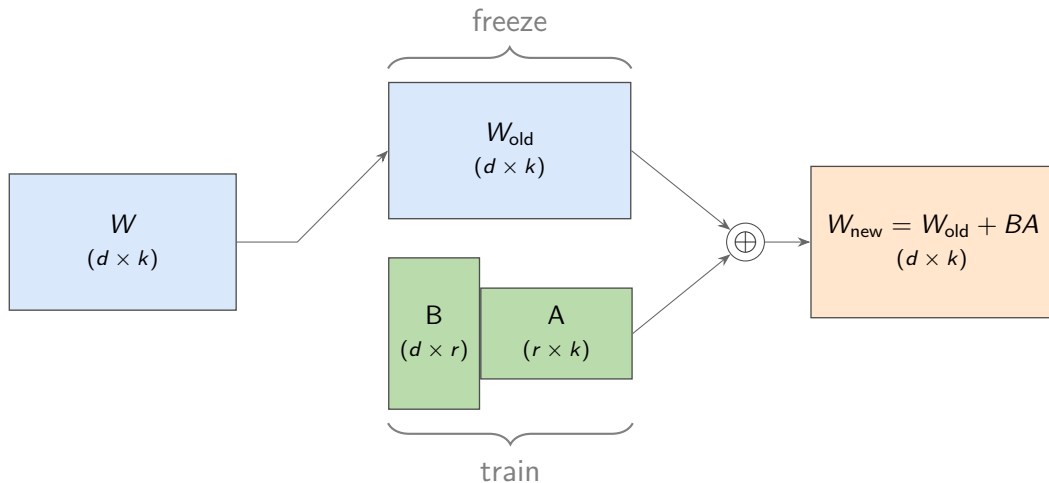
Updating just this *one* matrix requires storing over 1 million gradients in VRAM.

- ▶ **A “Friendly” Reminder:**
 - ▶ GPT-2 Medium (345M parameters) is considered *tiny* by modern standards.
 - ▶ W_q is only **one of hundreds** of matrices (across 24 layers).

How does LoRA work?

- ▶ **LoRA's idea** [Hu et al., 2022]:
 - ▶ Instead of updating the massive $W \in \mathbb{R}^{d \times k}$, we treat it as fixed “knowledge”
 - ▶ Introduce a trainable update: $\Delta W = B \times A$
 - ▶ $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$
 - ▶ The new updated W is $W_{new} = W + \Delta W$
- ▶ **When the magic happens:** Choose a small $r \ll d, k$!
- ▶ Instead of training $d \times k$ parameters of W , we train only B and A , meaning only $(d \times r) + (r \times k)$ parameters
- ▶ If $r = 8$ while $d = k = 1024$ (as in [GPT-2 Medium](#)), the number of trainable parameters is $1024 \times 8 + 8 \times 1024 = \mathbf{16,384} \ll 1,048,576$ (without LoRA)
 - 98.4% reduction in trainable parameters!
- ▶ r controls the **performance-cost** trade-off

Illustrate LoRA



Retrieval-Augmented Generation (RAG)

What is RAG? Why is RAG?

- ▶ Suppose we have a pre-trained LLM and want it to analyze a document (e.g., financial statements, earnings call transcripts), which is new to the model

What is RAG? Why is RAG?

- ▶ Suppose we have a pre-trained LLM and want it to analyze a document (e.g., financial statements, earnings call transcripts), which is new to the model
- ▶ Since most LLMs are **general-purpose**, their responses can be very general

What is RAG? Why is RAG?

- ▶ Suppose we have a pre-trained LLM and want it to analyze a document (e.g., financial statements, earnings call transcripts), which is new to the model
- ▶ Since most LLMs are **general-purpose**, their responses can be very general
- ▶ We must provide relevant information from the document at inference time to obtain accurate and domain-specific responses

What is RAG? Why is RAG?

- ▶ Suppose we have a pre-trained LLM and want it to analyze a document (e.g., financial statements, earnings call transcripts), which is new to the model
 - ▶ Since most LLMs are **general-purpose**, their responses can be very general
 - ▶ We must provide relevant information from the document at inference time to obtain accurate and domain-specific responses
 - ▶ **Retrieval-Augmented Generation (RAG)** is a framework that
 - ▶ **retrieves** relevant pieces of the document, and
 - ▶ **injects** (or *augments*) them into the prompt so the LLM can ground its answer in the source text
- RAG is a way to **contextualize** an LLM

What is RAG? Why is RAG?

- ▶ Suppose we have a pre-trained LLM and want it to analyze a document (e.g., financial statements, earnings call transcripts), which is new to the model
- ▶ Since most LLMs are **general-purpose**, their responses can be very general
- ▶ We must provide relevant information from the document at inference time to obtain accurate and domain-specific responses
- ▶ **Retrieval-Augmented Generation (RAG)** is a framework that
 - ▶ **retrieves** relevant pieces of the document, and
 - ▶ **injects** (or *augments*) them into the prompt so the LLM can ground its answer in the source text
- RAG is a way to **contextualize** an LLM
- ▶ Important: RAG $\neq$ LLMs. RAG is a system built around an LLM to contextualize its responses using external data

How to do RAG?

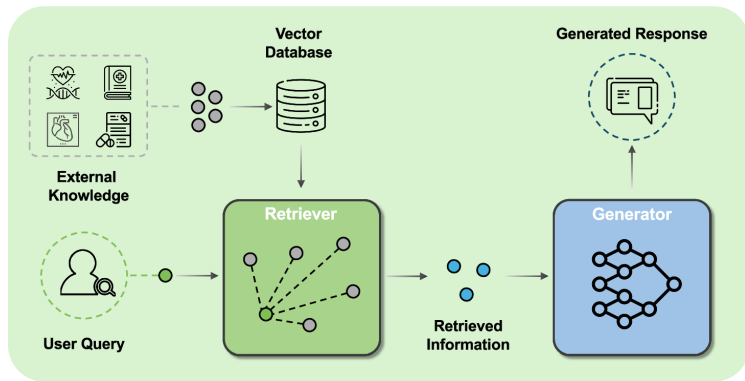


Figure: A RAG system. The image is from Yang et al. [2025].

- ▶ “Augmented” is easy, it’s usually in the following form: "`<user_query>`. Answer based on `<relevant_info_1>`, `<relevant_info_2>`"
- ▶ “Generation” is easier, that’s the LLM’s work
- ▶ “Retrieval”, i.e., finding `<relevant_info_1>`, is harder

Retrieval

- ▶ **Retrieval:** select pieces of information from your document (or knowledge database) that are **most relevant to your query**

Retrieval

- ▶ **Retrieval**: select pieces of information from your document (or knowledge database) that are **most relevant to your query**
- ▶ “most relevant to your query” $\approx$ “most **similar** to your query”

Retrieval

- ▶ **Retrieval**: select pieces of information from your document (or knowledge database) that are **most relevant to your query**
- ▶ “most relevant to your query” $\approx$ “most **similar** to your query”
- ▶ In other words, retrieval = finding document chunks that are **most similar** to your query

Retrieval

- ▶ **Retrieval**: select pieces of information from your document (or knowledge database) that are **most relevant to your query**
- ▶ “most relevant to your query” $\approx$ “most **similar** to your query”
- ▶ In other words, retrieval = finding document chunks that are **most similar** to your query
- ▶ Suppose your documents are split into M chunks (sentences, paragraphs, pages, sections, etc.), labeled $c_1, c_2, \dots, c_M$. Each chunk c_i is represented by a vector v_i using an embedding model (Doc2Vec, ELMo, BERT, GPT embeddings, etc.)

Retrieval

- ▶ **Retrieval**: select pieces of information from your document (or knowledge database) that are **most relevant to your query**
- ▶ “most relevant to your query” $\approx$ “most **similar** to your query”
- ▶ In other words, retrieval = finding document chunks that are **most similar** to your query
- ▶ Suppose your documents are split into M chunks (sentences, paragraphs, pages, sections, etc.), labeled $c_1, c_2, \dots, c_M$. Each chunk c_i is represented by a vector v_i using an embedding model (Doc2Vec, ELMo, BERT, GPT embeddings, etc.)
- ▶ Your query q is also represented as a vector v_q

Retrieval

- ▶ **Retrieval**: select pieces of information from your document (or knowledge database) that are **most relevant to your query**
- ▶ “most relevant to your query” $\approx$ “most **similar** to your query”
- ▶ In other words, retrieval = finding document chunks that are **most similar** to your query
- ▶ Suppose your documents are split into M chunks (sentences, paragraphs, pages, sections, etc.), labeled $c_1, c_2, \dots, c_M$. Each chunk c_i is represented by a vector v_i using an embedding model (Doc2Vec, ELMo, BERT, GPT embeddings, etc.)
- ▶ Your query q is also represented as a vector v_q
- ▶ Compute the similarity between the query and each chunk, call it $s_{q,i}$. The similarity can be cosine similarity, $1 - L_2$ distance, etc.

Retrieval

- ▶ **Retrieval**: select pieces of information from your document (or knowledge database) that are **most relevant to your query**
- ▶ “most relevant to your query” $\approx$ “most **similar** to your query”
- ▶ In other words, retrieval = finding document chunks that are **most similar** to your query
- ▶ Suppose your documents are split into M chunks (sentences, paragraphs, pages, sections, etc.), labeled $c_1, c_2, \dots, c_M$. Each chunk c_i is represented by a vector v_i using an embedding model (Doc2Vec, ELMo, BERT, GPT embeddings, etc.)
- ▶ Your query q is also represented as a vector v_q
- ▶ Compute the similarity between the query and each chunk, call it $s_{q,i}$. The similarity can be cosine similarity, $1 - L_2$ distance, etc.
- ▶ Retrieve the top K chunks with the highest $s_{q,i}$ to feed into the LLM

Remarks on retrieval

- ▶ When your documents or knowledge base are split into chunks and converted into vectors, these vectors are stored for later use in a **vector database**
- ▶ The most popular vector database is [FAISS](#) [Douce et al., 2024]
- ▶ For personal RAG applications, it is recommended to build your own vector database using your own data
- ▶ **Warning:** The embedding model used to construct the vector database **MUST BE IDENTICAL** to the one used to embed your queries

LLMs in financial research

Building finance-specific LLMs

- ▶ BloombergGPT Wu et al. [2023] is the first major attempt to build a finance-specific LLM; but ... it's closed-source
- ▶ Yang et al. [2023] respond to Bloomberg by introducing [FinGPT](#) and open-source it
- ▶ Xie et al. [2023] also introduce [FinMA-7B](#) based on LLaMA. There are other versions: [FinMA-7B-full](#) and FinMA-30B

Using LLMs in financial research

- ▶ Common trend: Using LLMs (especially ChatGPT) to measure text-based firm indicators, such as managers' outlook, risks, etc.
- ▶ Jha et al. [2024b] use ChatGPT 3.5 Turbo to extract the economic outlook expressed in US firms' conference call transcripts. Using the same technique and data, they [Jha et al., 2024a] measure firm-level managers' anticipated changes in capital expenditures
- ▶ Fan et al. [2025] measure firm-level supply chain risk using ChatGPT (GPT-4o mini)
- ▶ Chen et al. [2025] compare ChatGPT-3.5 and DeepSeek-R1 in predicting the stock market and the macroeconomy and find that ChatGPT wins

Challenges and threats of using LLMs in financial research

- ▶ LLM performance strongly depends on prompt design and model version, requiring careful engineering and validation
- ▶ LLM outputs may reflect model biases or training data artifacts, meaning rigorous economic validation is essential
- ▶ **Look-ahead Bias:** If an LLM was trained on data up to 2024, and you ask it to “predict” 2023 stock prices, it already knows the answer. This invalidates many “backtesting” papers

Coming next

Coming next

- ▶ Hands-on practice
 - ▶ Quantization and LoRA
 - ▶ [Notebook](#)/[Solution](#)/[RAG](#)
- ▶ In the second half
 - ▶ Guest lecture

Our guest



Taejin Park

- ▶ Head of Financial Markets Research Support at the Bank of International Settlements (BIS)
- ▶ Experience in OECD:
 - ▶ The Economics Department
 - ▶ The International Energy Agency
 - ▶ The Statistics and Data Directorate
- ▶ Main interest: Applications of AI technologies in economic research

References I

- Jian Chen, Guohao Tang, Guofu Zhou, and Wu Zhu. ChatGPT and DeepSeek: Can they predict the stock market and macroeconomy? [arXiv preprint arXiv:2502.10008](#), 2025.
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. 2024.
- Siyu Fan, Yifei Wu, and Ruochen Yang. Measuring firm-level supply chain risk using a generative large language model. [Finance Research Letters](#), 77:107111, 2025.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In [International Conference on Learning Representations](#), 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Manish Jha, Jialin Qian, Michael Weber, and Baozhong Yang. ChatGPT and corporate policies. Technical report, National Bureau of Economic Research, 2024a.
- Manish Jha, Jialin Qian, Michael Weber, and Baozhong Yang. Generative AI, managerial expectations, and economic activity. [arXiv preprint arXiv:2410.03897](#), 2024b.
- Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. BloombergGPT: A large language model for finance. [arXiv preprint arXiv:2303.17564](#), 2023.
- Qianqian Xie, Weiguang Han, Xiao Zhang, Yanzhao Lai, Min Peng, Alejandro Lopez-Lira, and Jimin Huang. PIXIU: A large language model, instruction data and evaluation benchmark for finance, 2023.
- Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. FinGPT: Open-source financial large language models. [arXiv preprint arXiv:2306.06031](#), 2023. URL <https://arxiv.org/abs/2306.06031>. First official FinGPT paper; FinLLM Workshop at IJCAI 2023.
- Rui Yang, Yilin Ning, Emilia Keppo, Mingxuan Liu, Chuan Hong, Danielle S Bitterman, Jasmine Chiat Ling Ong, Daniel Shu Wei Ting, and Nan Liu. Retrieval-augmented generation for generative artificial intelligence in health care. [Npj health systems](#), 2(1):2, 2025.